

Workflow Addendum - S-T6, O-T6, O-T7, O-T8, O-T9 Full File Ledger

Purpose: this addendum explains exactly what each late workflow step does, which tool owns it, what to upload to ChatGPT, what ChatGPT creates, what Codex must have in the project folder, and what Codex is allowed to create.

Canonical roots:

Root	Meaning
/workspace	Shared project workspace. Code, scripts, runs, smoke reports, generated batch folders.
/mnt/egress/dev-recordings/skeleton/<batch-slug>/	Skeleton batch evidence root.
/mnt/egress/organs/dev-recordings/organs/<batch-slug>/	Organ batch evidence root.
/mnt/ingress/infra/skeleton/companion/<batch-slug>/COMPANION.md	Skeleton companion output.
/mnt/ingress/infra/organs/companion/<batch-slug>/COMPANION.md	Organ companion output.
/workspace/docs/IDEMPOTENT_SMOKETEST.md	Smoke-test reference/specification document. Not normally uploaded every time.
/workspace/scripts/smoke_current_state.sh	Actual reusable smoke script that Codex runs.

Important distinction:

Thing	Used for
IDEMPOTENT_SMOKETEST.md	Manual/spec. Upload or reference only when creating, changing, or debugging the smoke script.
smoke_current_state.sh	Actual command runner. Codex executes this after batches and before/after config integration.

Overall order

Time	Phase	Main action	Smoke-test placement	Companion placement	Config integration placement
T1	Skeleton	Run skeleton batch 01	Run after batch	Optional if logical group boundary	Not yet
T2	Skeleton	Run skeleton batch 02	Run after batch	Optional if logical group boundary	Not yet
T3	Skeleton	Continue skeleton batches 03-24	Run after every batch	Update after logical groups	Not yet
T4	Skeleton complete	Validate all skeleton evidence	Run skeleton-complete smoke	Update final skeleton companion/index	Not yet
T5	Organs	Run organ batch R01	Run after batch	Optional if logical group boundary	Not yet
T6	Organs	Continue organ batches R02-R12	Run after every batch	Update after logical groups	Not yet
T7	Pre-config	Create integration manifest from completed evidence	Run pre-config smoke	Companions must be current enough	Prepare config integration
T8	Config batch creation	ChatGPT creates operator/vmuser config-integration batch	No code run unless asked	No companion update	Generate config Codex batch
T9	Config integration	Codex runs generated config integration as vmuser/operator	Run post-config smoke	No companion update unless docs changed	Update config/lv/workflow only here

What counts as a logical group for S-T6 and O-T6

A logical group is a stable checkpoint where the companion should be updated because the meaning of the platform changed enough that a future batch or operator needs readable documentation.

Skeleton logical groups

Group	Suggested batch range	Companion update recommended when	Why it is a group
S-G1 Runtime substrate	01-03	After basic folders, CLI, run layout, and metadata contract exist	Future work depends on the folder and command contract.
S-G2 Dummy science contract	04-06	After dummy simulator, dummy outputs, and report schema exist	Organ batches later must preserve these output contracts.
S-G3 Pipeline orchestration	07-10	After batch runner, status files, and smokeable pipeline path exist	This becomes the operational skeleton.
S-G4 Evidence and recording	11-14	After POSTCHECK and INTEGRATION_REQUEST behavior is stable	Operator/config integration depends on this evidence.
S-G5 Interfaces and validation	15-18	After schemas, validators, dry-run gates, and safety checks exist	Prevents organs from drifting away from skeleton contracts.
S-G6 Packaging and handoff	19-24	After final skeleton commands, docs, smoke, and handoff files are stable	This is the final skeleton baseline before organs.

Minimum rule: run smoke after every skeleton batch, then update skeleton companion after every logical group or after any contract-changing batch.

Organ logical groups

Group	Suggested batch range	Companion update recommended when	Why it is a group
O-G1 Real-organ adapter shell	R01-R02	After real-organ dry-run entry point exists	Establishes real-organ path without live risk.
O-G2 Real data/model wiring	R03-R05	After inputs, parsers, model adapters, and dry-run outputs exist	Documents real science inputs while preserving skeleton output contract.
O-G3 Real simulation/reporting	R06-R08	After real report generation and output comparison exist	Captures what real organs now produce and how to verify them.
O-G4 Safety and gating	R09-R10	After guarded live gates, env vars, and safety checks exist	Prevents accidental live/expensive operations.
O-G5 Final organ handoff	R11-R12	After final organ smoke, docs, evidence, and integration requests are stable	This is the organ baseline used for config integration.

Minimum rule: run smoke after every organ batch, then update organ companion after every logical group or after any real-output/safety-contract change.

Step S-T6 - Update skeleton companion after a checked skeleton state

Field	Exact content
Step owner	Usually ChatGPT drafts the companion content; Codex may write it to the mounted ingress folder.
When	After a skeleton logical group, after a contract-changing skeleton batch, or after skeleton-complete smoke passes/warns acceptably.
ChatGPT prompt	Update the skeleton companion for the checked skeleton state. Use the latest POSTCHECK.md, INTEGRATION_REQUEST.md, smoke report, current codebase analysis, and existing companion docs. Preserve the posthoc config-integration model. Do not invent missing files. Return updated COMPANION.md content and list any missing required evidence.
Upload to ChatGPT	COMPANION_GENERATOR_INSTRUCTIONS.md; latest POSTCHECK.md; latest INTEGRATION_REQUEST.md; latest SMOKE_REPORT.md; latest codebase analysis output; existing skeleton COMPANION.md if present; existing skeleton INDEX.md if present; relevant generated batch SPEC.md and RUN_INSTRUCTIONS.md if the companion needs exact commands.
ChatGPT creates	Updated COMPANION.md content; optional INDEX.md update text; optional missing-file report; optional checklist of commands/contracts documented.
Codex must have access to	/workspace; /mnt/egress/dev-recordings/skeleton/<batch-slug>/POSTCHECK.md; /mnt/egress/dev-recordings/skeleton/<batch-slug>/INTEGRATION_REQUEST.md; latest smoke report under /workspace/runs/smoke/.../SMOKE_REPORT.md; /mnt/ingress/infra/skeleton/companion/<batch-slug>/; ChatGPT-created companion content or patch.

Field	Exact content
Codex prompt	Write the provided skeleton companion update to /mnt/ingress/infra/skeleton/companion/<batch-slug>/COMPANION.md. Update INDEX.md only if provided. Do not edit config. Do not run live actions. Preserve existing companion history unless replacement was explicitly provided.
Codex may run	File write commands only; optional local existence checks such as test -f; optional smoke command only if explicitly requested.
Codex creates/updates	/mnt/ingress/infra/skeleton/companion/<batch-slug>/COMPANION.md; optional /mnt/ingress/infra/skeleton/companion/INDEX.md; optional write log.
Codex must not create	Config/lv/workflow edits; organ companion docs; invented integration manifests.

Step O-T6 - Update organ companion after a checked organ state

Field	Exact content
Step owner	Usually ChatGPT drafts the organ companion content; Codex may write it to the mounted ingress folder.
When	After an organ logical group, after an organ safety/output contract changes, or after organ-complete smoke passes/warns acceptably.
ChatGPT prompt	Update the organ companion for the checked real-organ state. Use the latest organ POSTCHECK.md, INTEGRATION_REQUEST.md, smoke report, current codebase analysis, existing organ companion docs, and the relevant skeleton companion contract. Preserve skeleton output contracts. Do not invent missing files. Return updated COMPANION.md content and list any missing required evidence.
Upload to ChatGPT	ORGAN_COMPANION_GENERATOR_INSTRUCTIONS.md; latest organ POSTCHECK.md; latest organ INTEGRATION_REQUEST.md; latest organ SMOKE_REPORT.md; latest codebase analysis output; existing organ COMPANION.md if present; existing organ INDEX.md if present; relevant skeleton COMPANION.md or skeleton contract summary; relevant organ batch SPEC.md and RUN_INSTRUCTIONS.md if exact commands are needed.
ChatGPT creates	Updated organ COMPANION.md content; optional organ INDEX.md update text; optional missing-file report; optional safety-gate checklist.
Codex must have access to	/workspace; /mnt/egress/organs/dev-recordings/organs/<batch-slug>/POSTCHECK.md; /mnt/egress/organs/dev-recordings/organs/<batch-slug>/INTEGRATION_REQUEST.md; latest smoke report under /workspace/runs/smoke/.../SMOKE_REPORT.md; /mnt/ingress/infra/organs/companion/<batch-slug>; ChatGPT-created companion content or patch.
Codex prompt	Write the provided organ companion update to /mnt/ingress/infra/organs/companion/<batch-slug>/COMPANION.md. Update INDEX.md only if provided. Do not edit config. Do not run live/guarded-real actions unless explicitly requested. Preserve existing companion history unless replacement was explicitly provided.
Codex may run	File write commands only; optional local existence checks such as test -f; optional dry-run/smoke command only if explicitly requested.
Codex creates/updates	/mnt/ingress/infra/organs/companion/<batch-slug>/COMPANION.md; optional /mnt/ingress/infra/organs/companion/INDEX.md; optional write log.
Codex must not create	Config/lv/workflow edits; skeleton companion docs; invented integration manifests.

Step O-T7 - ChatGPT creates the integration manifest from completed evidence

Field	Exact content
Step owner	ChatGPT. Codex should not integrate config yet.
When	After all required skeleton and organ evidence exists and pre-config smoke has been run.
ChatGPT prompt	Create INTEGRATION_MANIFEST.md from the completed skeleton and organ evidence. Read every supplied INTEGRATION_REQUEST.md, POSTCHECK.md, smoke report, and COMPANION.md. Classify each integration item as required, optional, deferred, or blocked. Do not edit config. Do not invent missing files. If required files are missing, stop and list them.
Upload to ChatGPT	INTEGRATION_MANIFEST_TEMPLATE.md; 01_CHATGPT_INTEGRATION_PLANNER.md; CONFIG_TOOL.md; optional config_platform_integration_bridge_plan.md; all required skeleton INTEGRATION_REQUEST.md files; all required skeleton POSTCHECK.md files; final skeleton companion(s); all required organ INTEGRATION_REQUEST.md files; all required organ POSTCHECK.md files; final organ companion(s); latest pre-config SMOKE_REPORT.md; current config/codebase analysis if available.
ChatGPT creates	INTEGRATION_MANIFEST.md; missing-file report if needed; blocked/deferred list; config-integration scope summary; manifest slice plan for Codex.
Codex must have access to	Nothing required for this step unless ChatGPT asks Codex to collect files. If used for collection, Codex only reads/copies evidence files from /mnt/egress/.../mnt/ingress/... and /workspace/runs/smoke/...

Field	Exact content
Codex prompt if used only for collection	Collect the listed evidence files into a review bundle. Do not modify code or config. Do not create INTEGRATION_MANIFEST.md yourself unless explicitly instructed with a template.
Codex may run	Read-only find, ls, cat, archive/copy commands for evidence collection.
Codex creates/updates	Optional evidence bundle only, for example /workspace/handoff/config_integration_evidence_bundle/.
Codex must not create	Config changes; lv profiles; aliases; workflow steps; health checks; bootstrap rows.

Step O-T8 - ChatGPT creates the operator/vmuser config-integration Codex batch

Field	Exact content
Step owner	ChatGPT creates the batch; Codex runs it later in O-T9.
When	After INTEGRATION_MANIFEST.md is complete and accepted.
ChatGPT prompt	Generate a vmuser/operator config-integration Codex batch from INTEGRATION_MANIFEST.md. The batch must update config/lv/workflow integration only. It must not modify science implementation code except where explicitly required for a launcher or health-check reference. Include CODEX_PROMPT.txt, PROJECT_CACHE.md, SPEC.md, RUN_INSTRUCTIONS.md, POSTCHECK_TEMPLATE.md, and a manifest slice. Include safe preflight and post-config smoke instructions.
Upload to ChatGPT	Final INTEGRATION_MANIFEST.md; 02_CODEX_OPERATOR_EXECUTION.md; CONFIG_TOOL.md; current config repository analysis or relevant config files; optional INTEGRATION_MANIFEST_TEMPLATE.md for structure checking; latest pre-config SMOKE_REPORT.md; relevant companion docs if commands/aliases need human explanation.
ChatGPT creates	codex_config_integration_batch_<date-or-scope>.zip; inside it: CODEX_PROMPT.txt, PROJECT_CACHE.md, SPEC.md, RUN_INSTRUCTIONS.md, POSTCHECK_TEMPLATE.md, INTEGRATION_MANIFEST_SLICE.md, optional README.md.
Codex must have access to	No execution yet. For packaging only, Codex is not required. If Codex is used to unzip/check the package, it needs the generated zip and a temp folder.
Codex prompt if used only to inspect package	Unzip and list the generated config-integration batch. Do not execute it. Verify required files exist: CODEX_PROMPT.txt, PROJECT_CACHE.md, SPEC.md, RUN_INSTRUCTIONS.md, POSTCHECK_TEMPLATE.md, INTEGRATION_MANIFEST_SLICE.md.
Codex may run	unzip -l, sha256sum, test -f, read-only inspection.
Codex creates/updates	Optional package listing or checksum file.
Codex must not create	Actual config edits during O-T8. Those belong to O-T9.

Step O-T9 - Codex runs vmuser/operator config integration

Field	Exact content
Step owner	Codex running as vmuser/operator. ChatGPT only reviews or prepares prompts.
When	After O-T8 batch is accepted and the operator role has access to the config repo and required evidence.
ChatGPT prompt to prepare Codex instruction	Prepare the exact Codex prompt for running the accepted config-integration batch as vmuser/operator. Include required files, forbidden actions, preflight checks, postcheck requirements, and post-config smoke command. Do not expand scope beyond INTEGRATION_MANIFEST_SLICE.md.
Upload to ChatGPT if asking for review	Accepted config batch zip or extracted files; INTEGRATION_MANIFEST.md; INTEGRATION_MANIFEST_SLICE.md; CONFIG_TOOL.md; current config repo analysis; previous config POSTCHECK if any; latest pre-config SMOKE_REPORT.md.
ChatGPT creates	Final Codex run prompt; optional run checklist; optional review of generated config batch before execution.
Codex must have access to	Extracted config-integration batch files: CODEX_PROMPT.txt, PROJECT_CACHE.md, SPEC.md, RUN_INSTRUCTIONS.md, POSTCHECK_TEMPLATE.md, INTEGRATION_MANIFEST_SLICE.md; config repo/tool files; /workspace/scripts/smoke_current_state.sh; /workspace/docs/IDEMPOTENT_SMOKETEST.md only if script behavior needs reference; relevant evidence files from /mnt/egress/...; relevant companions from /mnt/ingress/...

Field	Exact content
Codex prompt	Run the accepted vmuser/operator config-integration batch exactly as specified by SPEC.md and RUN_INSTRUCTIONS.md. Scope is limited to INTEGRATION_MANIFEST_SLICE.md. Perform preflight checks first. Update config/lv/workflow integration only. Do not alter skeleton/organ implementation code unless explicitly required by the manifest slice. Run post-config smoke. Write POSTCHECK.md and list every changed file.
Codex may run	Commands in RUN_INSTRUCTIONS.md; safe preflight checks; config-tool validation commands; bash /workspace/scripts/smoke_current_state.sh post-config; read-only checks against evidence/companion files.
Codex creates/updates	Config workflow step names; lv profiles; role aliases; health checks; bootstrap/install rows; status or launcher commands if manifest requires them; config integration POSTCHECK.md; changed-file list; post-config smoke report under /workspace/runs/smoke/.../SMOKE_REPORT.md.
Codex must not create	New science organs; new skeleton batches; unrequested live actions; config items not backed by the manifest; invented env vars without manifest evidence.

Quick command ledger

Situation	Command or instruction
After each skeleton batch	BATCH_SLUG=" <code><skeleton-batch-slug></code> " bash /workspace/scripts/smoke_current_state.sh skeleton-progress
After all skeleton batches	bash /workspace/scripts/smoke_current_state.sh skeleton-complete
After each organ batch	BATCH_SLUG=" <code><organ-batch-slug></code> " bash /workspace/scripts/smoke_current_state.sh organ-progress
Before config integration	bash /workspace/scripts/smoke_current_state.sh pre-config
After config integration	bash /workspace/scripts/smoke_current_state.sh post-config
Create or repair smoke script	Upload/reference /workspace/docs/IDEMPOTENT_SMOKETEST.md and ask ChatGPT/Codex to create or repair /workspace/scripts/smoke_current_state.sh.

File tracking checklist by step

Step	Upload to ChatGPT	ChatGPT creates	Codex needs in project/access	Codex creates
S-T6	Generator instructions, POSTCHECK, INTEGRATION_REQUEST, SMOKE_REPORT, codebase analysis, existing skeleton companion/index, relevant SPEC/RUN_INSTRUCTIONS	Skeleton COMPANION.md content, optional INDEX update, missing-file list	/workspace, skeleton egress evidence, smoke report, skeleton ingress companion folder, ChatGPT output	Skeleton COMPANION.md, optional skeleton INDEX.md, optional write log
O-T6	Organ generator instructions, organ POSTCHECK, organ INTEGRATION_REQUEST, organ SMOKE_REPORT, codebase analysis, existing organ companion/index, skeleton companion/contract, relevant SPEC/RUN_INSTRUCTIONS	Organ COMPANION.md content, optional INDEX update, missing-file list, safety checklist	/workspace, organ egress evidence, smoke report, organ ingress companion folder, ChatGPT output	Organ COMPANION.md, optional organ INDEX.md, optional write log
O-T7	Manifest template, planner instructions, CONFIG_TOOL, bridge plan, all integration requests, postchecks, companions, pre-config smoke, config/code analysis	INTEGRATION_MANIFEST.md, missing-file report, blocked/deferred list, scope summary	Optional read-only access to collect evidence	Optional evidence bundle only
O-T8	Final manifest, operator execution instructions, CONFIG_TOOL, config repo analysis/files, pre-config smoke, relevant companions	Config-integration Codex batch zip and internal files	Optional generated zip for inspection only	Optional package listing/checksum only
O-T9	Accepted config batch, manifest/slice, CONFIG_TOOL, config repo analysis, previous config POSTCHECK, pre-config smoke if asking ChatGPT for review	Final Codex prompt/checklist/review	Extracted config batch, config repo/tool, smoke script, evidence, companions	Config/lv/workflow changes, health checks, aliases, bootstrap rows, POSTCHECK.md, changed-file list, post-config smoke report

Minimal prompts to copy

S-T6 prompt to ChatGPT

Update the skeleton companion for this checked skeleton state.
Use the uploaded POSTCHECK.md, INTEGRATION_REQUEST.md, SMOKE_REPORT.md, codebase analysis, existing COMPANION.md/INDEX.md, and relevant SPEC/RUN_INSTRUCTIONS.
Return updated COMPANION.md content and any INDEX.md update.
Do not invent missing files. If required evidence is missing, stop and list it.
Do not propose config edits here.

O-T6 prompt to ChatGPT

Update the organ companion for this checked real-organ state.
Use the uploaded organ POSTCHECK.md, INTEGRATION_REQUEST.md, SMOKE_REPORT.md, codebase analysis, existing organ COMPANION.md/INDEX.md, and relevant skeleton companion contract.
Return updated COMPANION.md content and any INDEX.md update.
Do not invent missing files. If required evidence is missing, stop and list it.
Do not propose config edits here.

O-T7 prompt to ChatGPT

Create INTEGRATION_MANIFEST.md from the completed skeleton and organ evidence.
Use all uploaded INTEGRATION_REQUEST.md, POSTCHECK.md, SMOKE_REPORT.md, COMPANION.md, CONFIG_TOOL.md, and the manifest template.
Classify each item as required, optional, deferred, or blocked.
If required evidence is missing, stop and list the exact missing files.
Do not write config integration steps yet except as planned manifest entries.

O-T8 prompt to ChatGPT

Generate a vmuser/operator config-integration Codex batch from the accepted INTEGRATION_MANIFEST.md.
Create CODEX_PROMPT.txt, PROJECT_CACHE.md, SPEC.md, RUN_INSTRUCTIONS.md, POSTCHECK_TEMPLATE.md, and INTEGRATION_MANIFEST_SLICE.md.
The batch may update config/lv/workflow integration only.
Include preflight, validation, changed-file list, and post-config smoke instructions.

O-T9 prompt to Codex

Run the accepted vmuser/operator config-integration batch exactly as specified.
Use CODEX_PROMPT.txt, PROJECT_CACHE.md, SPEC.md, RUN_INSTRUCTIONS.md, POSTCHECK_TEMPLATE.md, and INTEGRATION_MANIFEST_SLICE.md.
Scope is limited to INTEGRATION_MANIFEST_SLICE.md.
Run preflight checks first.
Update config/lv/workflow integration only.
Do not alter skeleton/organ implementation code unless explicitly required by the manifest slice.
Run post-config smoke:

```
bash /workspace/scripts/smoke_current_state.sh post-config
```

Write POSTCHECK.md, include the smoke report path, and list every changed file.